STATE / INTERLEAVING / PROOF

Race Conditions Explained

Find the race window, prove the effect,
and repair the rule.



Start with a rule. Follow two requests through it.
Prove what actually changed, then test the repair.
Practice with real HTTP and a shared local database.

Haroon Shahid

First edition / Version 1.0 / 4 November 2026

Before you begin

Race Conditions Explained

Find the race window, prove the effect, and repair the rule

By Haroon Shahid. First edition, version 1.0. 4 November 2026.

Copyright 2026 Haroon Shahid. The purchased publication is licensed for personal use under 05-ABOUT/Publication_License.txt. The separately identified lab code has its own license. Prepared with AI assistance. For corrections: gaknt2@gmail.com. The free preview may be shared unchanged with attribution. No resale or public redistribution of the paid publication is granted.

This book teaches responsible analysis through fictional examples and bounded local practice. It does not certify expertise. Use only systems and accounts you own or are explicitly authorized to assess.

The validation report identifies executed tests and limits. Framework and protocol guidance is version-sensitive. The local lab uses real HTTP, independent worker threads, a shared SQLite database, and explicitly controlled schedules with fictional data. It is not a production security library.

PDF code can soft-wrap for page fit. The offline HTML and source preserve original lines. Responsive HTML and scalable diagrams provide an alternative reading format; no full PDF accessibility conformance is claimed.

Free preview: introduction, roadmap and Chapters 1-3. The full package adds the complete curriculum, answered reviews, workbook, reference sheets and Interleave Lab Lab.

Contents

Before you begin 2
Contents 3
Introduction 4
Learning roadmap 5
01. The rule the application must keep 7
02. Overlap without simultaneous execution 9
03. Follow a request to durable state 11
Appendix A. Answer explanations 13
Sources and further reading 14

Try this while you read

After Chapter 3, draw two requests on separate lines. Mark what each reads, what changes, and what you would check after both requests finish. The full workbook turns that sketch into the first three coupon experiments: sequential, overlapping, and repaired.

Introduction

A shop has one coupon left. Two requests arrive. Both read “one.” One finishes and saves “zero.” The other also saves “zero.” The counter looks right. The ledger shows two credits.

That is the kind of race this book teaches you to see. We will follow fictional Harbor Market from a browser request to a shared database, through a queue, and into a protected state. You will learn to say what the application promised, which two operations can interfere, what actually happened, and how the state owner can prevent it.

The method

Rule. Schedule. Effect. Repair. Write the rule in one sentence. Draw the order in which the operations read and write. Prove the effect with a ledger, a version or an independent protected read. Rerun that schedule against a repair and show the legitimate path still works.

The lab pauses real HTTP handlers at a named point. Two independent Node worker threads use separate connections to the same SQLite database. You can inspect the state before releasing both requests, then compare the final ledger. This is a controlled experiment. The barrier does not prove that a production server has the same window.

How to read this book

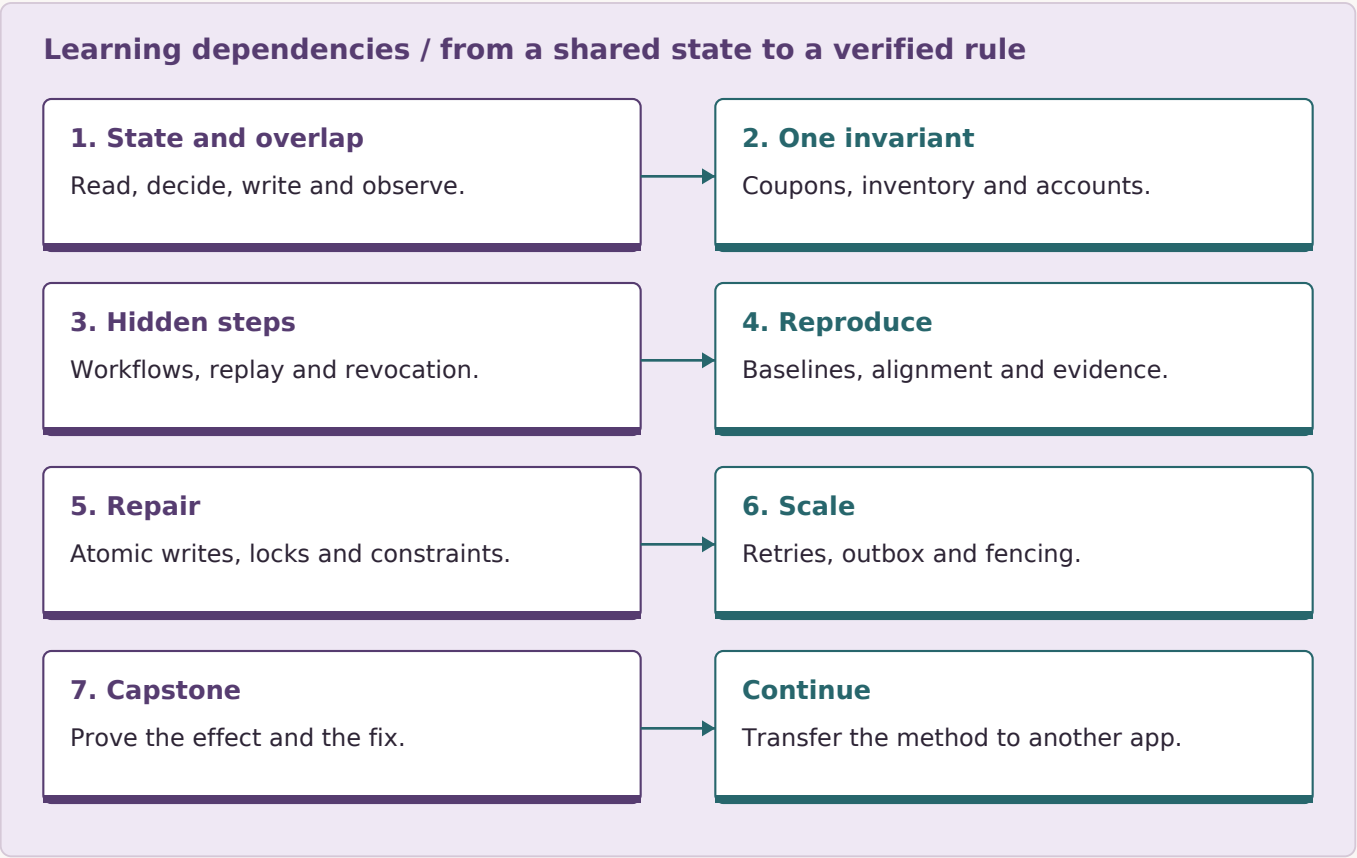
No concurrency background is assumed. Start with Chapters 1–6 if a database, session or commit is new to you. If you already test web apps, begin with the coupon, hidden login phase and checkout cases, then return to the foundations when needed. The workbook contains commands, predictions, results and a report worksheet.

Learning roadmap

Route	Read first	Show that you can...
First-time learner	Chapters 1–12	Explain overlap, a shared key and one durable effect.
Web security tester	Chapters 13–24	Predict a collision, capture controls and prove a bounded violation.
Builder or reviewer	Chapters 25–36	Select a guard, test multiple callers and reason about retries and failures.

Gate	Observable milestone
Foundation	Draw a safe order and an unsafe order for one coupon.
Practitioner	Compare sequential, overlapping and repaired schedules.
Advanced	Prove an intermediate-state and a cross-endpoint failure independently.
System design	Defend an invariant across a real state owner, retries and worker boundaries.
Transfer	Rebuild the method for an unfamiliar feature without copying a payload.

Each chapter has a mechanism diagram, a compact revision diagram, a practice prompt, three answered checks and a worked example. Read the first part in order. Afterwards, use the decision library and workbook by topic.



Read each row left to right, then move down. Each later stage uses the earlier mental model.

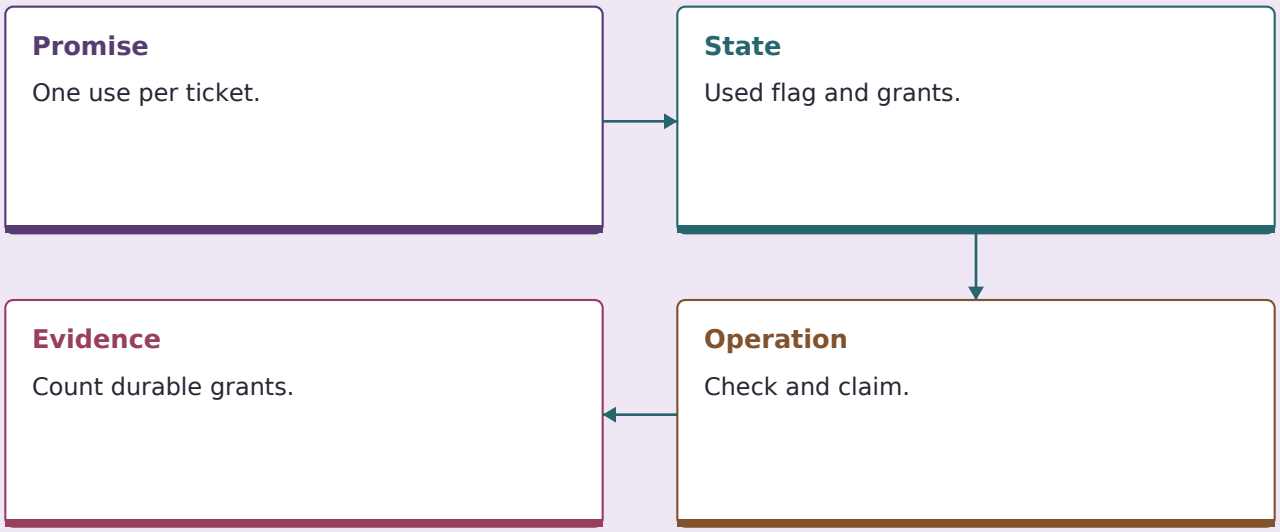
I. UNDERSTAND STATE AND OVERLAP

01. The rule the application must keep

Learn to: Describe shared state and turn a business promise into a testable rule.

Think of it this way: A cloakroom ticket may be used once. Two desks must agree about whether it has been used.

01A / From a promise to an observation



A testable rule tells you what final evidence to collect.

From a request to a rule

Harbor Market is a fictional shop. It gives Ada a discount code that works once. The server must record both the code's claim and the benefit. A request is a message asking the server to do that work. A response is what the server reports. **State** is information the server remembers, such as the claim, a balance, or a record in a database.

An **invariant** is a rule that should hold at a promised boundary. “One discount grant per account and campaign” can be checked against a ledger. “The API returns a friendly message” is not that rule. A log of two attempted requests also does not tell you whether there were two durable grants.

Start with the right key

Ask whether the code is single-use globally, once per account, or once per order. Those are three different promises. The key might be the code alone, an account and campaign, or an order and code. If the rule is vague, a tester may call a permitted second purchase a bug or overlook a collision across two orders.

The same approach works with inventory, funds, sessions, invitations, and one-time codes. Name the resource, the actor, the rule, and the authoritative record. Then observe the record after all operations finish.

Practice

Write two versions of “one coupon per customer.” Decide whether two browser tabs, two accounts, and two orders share the same claim key.

01B / State, schedule, guard, evidence

State	Schedule	Guard	Evidence
One unused ticket	Two requests	One committed claim	One grant total

Key takeaway. Start with the business rule and the state that proves whether it held.

Check your understanding

1.1 What is shared state?	**1.2** What is an invariant?	**1.3** Is a 200 response a durable grant?

Answer explanations: Appendix A. Sources: R01, R03, R04

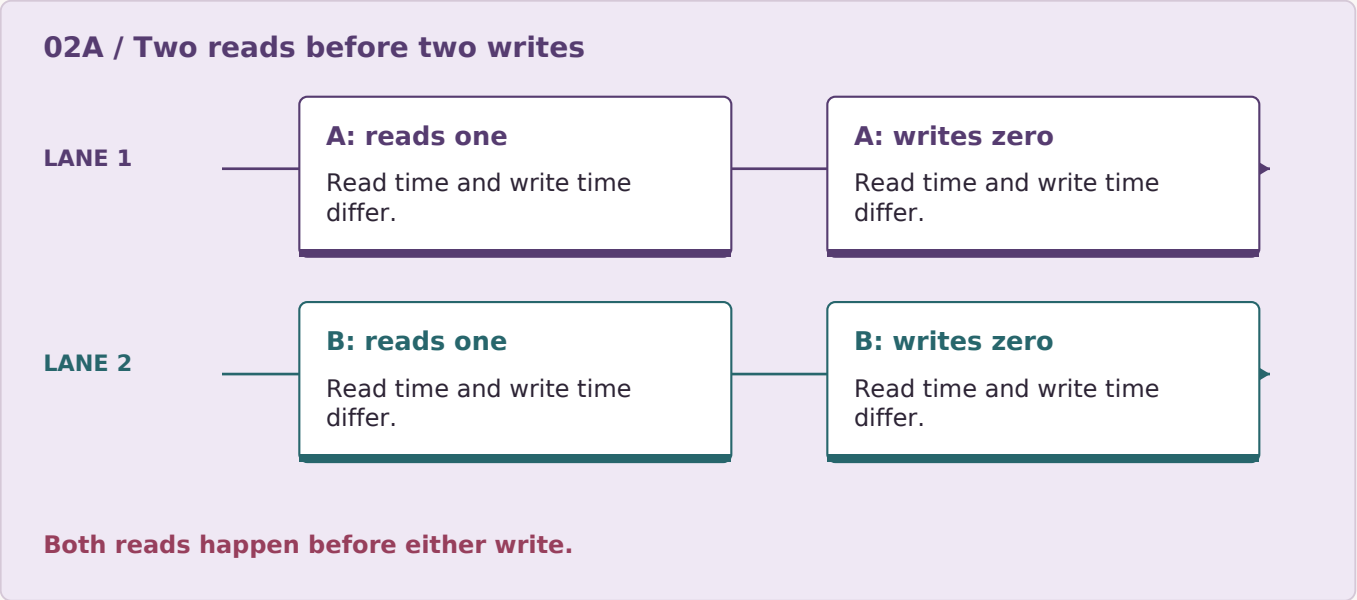
WORKED EXAMPLE / Choose the protected key

Ada uses a voucher on order X, then on order Y. If the promise is once per order, both may be valid. If it is once per campaign per account, the second violates the promise. The ledger query has to group by the correct key before a two-row count becomes proof.

02. Overlap without simultaneous execution

Learn to: Explain concurrency, parallelism, interleaving and an asynchronous pause.

Think of it this way: One cook can start boiling water and chop vegetables before the water finishes. Both jobs are in progress even with one cook.



Two requests can overlap even when each instruction runs separately.

Read the schedule

A request handler can read an unused code, wait for a database or service, and resume later. During the wait, another handler reads the same code. Both reads were true when performed. The defect is using that earlier truth as if it were still exclusive permission to write.

Concurrency means tasks make progress over overlapping periods. Parallelism means some execution actually occurs at the same time. Threads and separate worker processes can run in parallel. A single event loop can still allow asynchronous interleavings. A database might serialize the writes while the application still makes both decisions on old information.

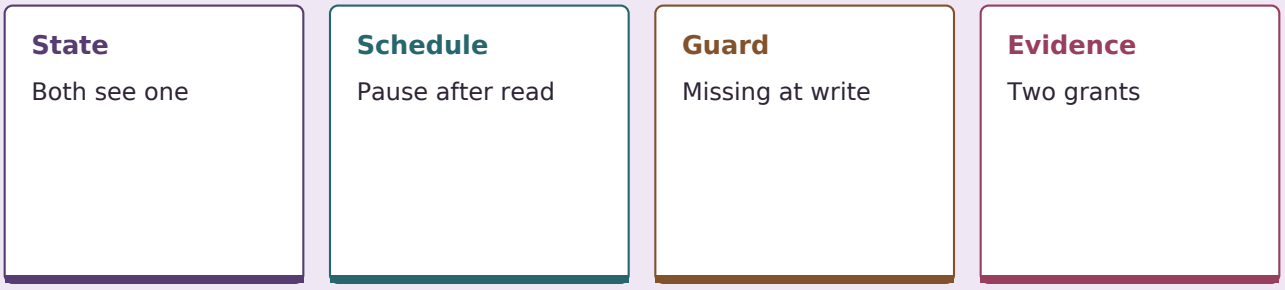
Four moves worth learning

Read the state. Decide if the operation is allowed. Change the state. Observe what became durable. Draw one row for each request and one column for each key step. Draw a barrier after both reads if you want to examine the risky order. The lab adds such a barrier for learning; real systems may expose the gap naturally through I/O, a separate query, or a service call.

Practice

Draw a safe schedule and a weak schedule for two attempts to redeem the last ticket. Label the first point at which the operation could safely report success.

02B / State, schedule, guard, evidence



Key takeaway. A race needs an unsafe ordering; it does not need two instructions to run at the same instant.

Check your understanding

2.1 Can one event loop have a race?	**2.2** Does Promise.all make database commits simultaneous?	**2.3** What is an interleaving?

Answer explanations: Appendix A. Sources: R01, R03, R14

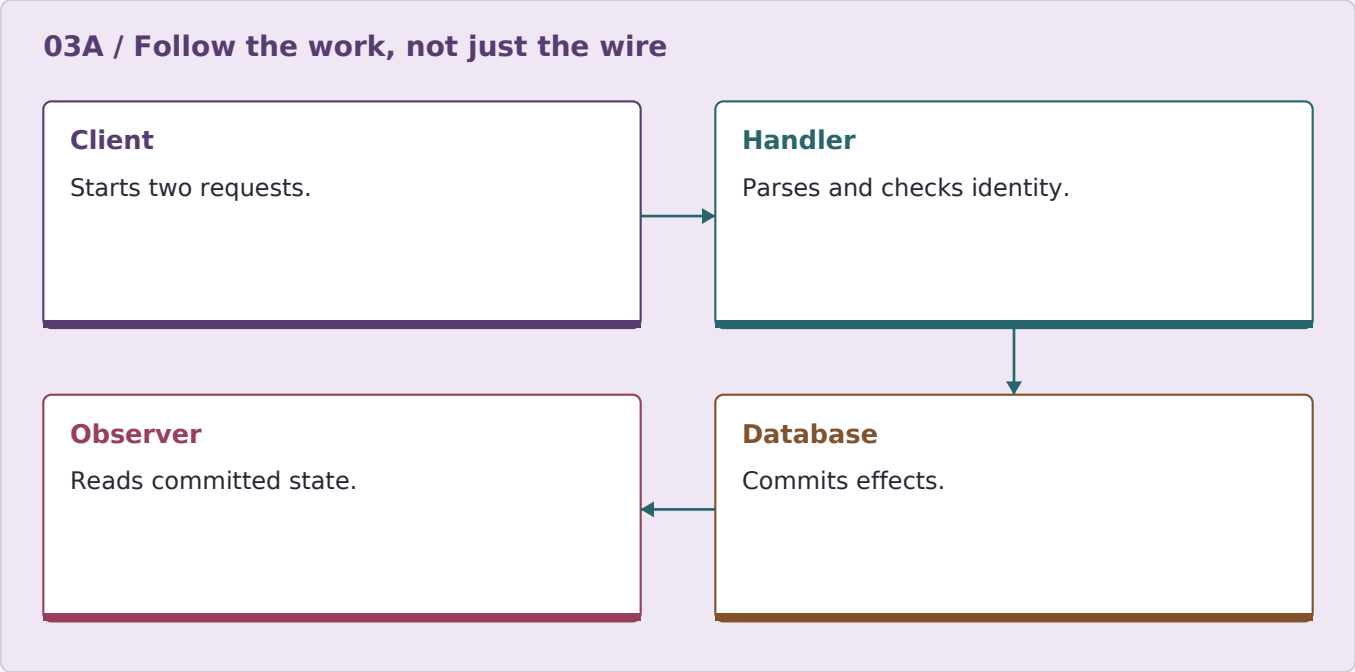
WORKED EXAMPLE / Order matters without simultaneous writes

A waits on a socket after reading one. B reads one while A is waiting. The database then writes A and B serially. The database did not execute both writes at once, yet the business decisions overlapped. Check the read-to-write order before looking for literal simultaneity.

03. Follow a request to durable state

Learn to: Separate client send time, server processing, database commit and delivery.

Think of it this way: Mailing two letters together does not tell you which clerk opens or files either one first.



The network, application, database and worker may all have different queues.

One request, several clocks

A client sends an HTTP request over a connection. A proxy may buffer it or hand it to an application instance. The handler can perform a read, start a transaction, and wait for another system. A transaction's **commit** makes its changes visible under the database's rules. A response can leave later. If delivery fails, the server may still have committed.

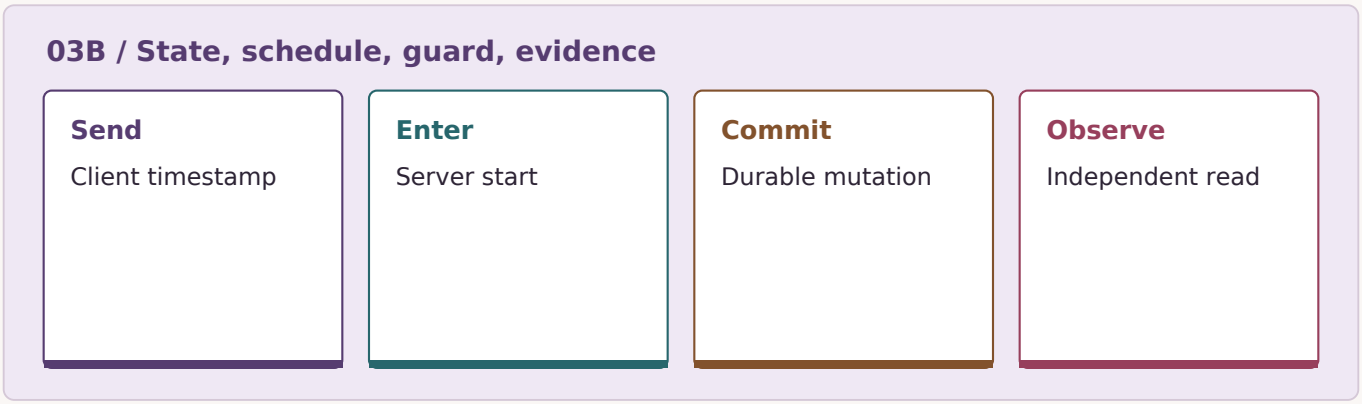
Client timestamps show what happened at the client. They do not, by themselves, show when a worker read the record or wrote it. Queue timestamps help, but clocks from different machines may not agree closely enough to establish a tiny gap. Request IDs, transaction IDs and ordered evidence from the state owner are stronger than comparing two unrelated wall clocks.

The observation path

For the shop's demo coupon, capture the response pair and then ask the database-backed endpoint for the final number of grants. The test harness records which worker paused and resumed. It does not infer a commit from “request started.” When reviewing a real asynchronous flow, distinguish accepted, committed, delivered and applied. A 202 usually acknowledges acceptance, not completion.

Practice

Sketch a request that commits and then times out before its response arrives. Explain how a retry could duplicate the effect and how you would verify it.



Key takeaway. A wire trace can prove arrival behavior; a ledger or state read proves the business effect.

Check your understanding

3.1 Can a response be lost after a commit?	**3.2** Why can one incoming request cause several background operations?	**3.3** Which layer owns whether an inventory count actually fell?

Answer explanations: Appendix A. Sources: R04, R05, R09, R13

WORKED EXAMPLE / Lost acknowledgment

An order request times out; the ledger already contains order 17. A retry returns a new order ID, 18. The timeout itself was not a rollback. If the safe retry returns order 17 and there is only one durable record, the repeated response is expected.

Appendix A. Answer explanations

Chapter 01: The rule the application must keep

1.1 What is shared state? Information that two operations can read or change, such as a balance or coupon row.

1.2 What is an invariant? A rule that must hold at the points where the application promises it, such as at most one grant after commits.

1.3 Is a 200 response a durable grant? No. Verify the final ledger or other authoritative record.

Chapter 02: Overlap without simultaneous execution

2.1 Can one event loop have a race? Yes. One handler can await I/O while another handler acts on the same state.

2.2 Does Promise.all make database commits simultaneous? No. It starts or waits on asynchronous work; the server and database still choose execution order.

2.3 What is an interleaving? The combined order of meaningful steps taken by overlapping operations.

Chapter 03: Follow a request to durable state

3.1 Can a response be lost after a commit? Yes. A timeout does not mean the server rolled back.

3.2 Why can one incoming request cause several background operations? The handler may queue work, call services, or write an event for later processing.

3.3 Which layer owns whether an inventory count actually fell? The authoritative inventory store and its committed state, not the client's spinner.

Sources and further reading

Research cutoff: 4 November 2026. The catalog distinguishes consulted material from targeted further reading. It does not claim every linked page was audited end to end or every client was deployed. Live documents change. Historical fixes are not current release recommendations. Original diagrams and synthetic exercises do not imply vendor endorsement.

R01 - CWE-362: Concurrent Execution Using Shared Resource

Consulted. General race weakness, shared resources and coordination.
https://cwe.mitre.org/data/definitions/362.html

R03 - OWASP Race Conditions

Consulted. Web testing, source review and common remediation patterns.
https://community.owasp.org/pages/vulnerabilities/race_conditions

R04 - PortSwigger: Race conditions

Consulted. Limit overruns, hidden states, session locking, multi-endpoint timing and alignment.
https://portswigger.net/web-security/race-conditions

R05 - Smashing the State Machine

Consulted. Published research on hidden states and reducing network jitter.
https://portswigger.net/research/smashing-the-state-machine

R09 - HTTP Semantics, RFC 9110

Consulted. Method idempotency, preconditions and response semantics.
https://www.rfc-editor.org/rfc/rfc9110.html

R13 - Caido Replay documentation

Further reading. Use your installed tool version to inspect and replay scoped requests.
https://docs.caido.io/app/quickstart/replay

R14 - Node.js Event Loop

Consulted. Asynchronous callbacks and timing in a JavaScript runtime.
https://nodejs.org/learn/asynchronous-work/event-loop-timers-and-nexttick